Efficient Image Super-Resolution with Lightweight CNNs: A CPU-Friendly Adaptation of IMDN

Farida Gaber
School of Computing
Queen's University
Kingston, Canada
25nplx@queensu.ca

Raneem Alnaghy
School of Computing
Queen's University
Kingston, Canada
25htc5@queensu.ca

Rodina Khallaf
School of Computing
Queen's University
Kingston, Canada
25cfsf@queensu.ca

***Abstract*—Single-image super-resolution (SISR) aims to reconstruct high-resolution images from low-resolution inputs, with applications spanning mobile photography, medical imaging, and satellite remote sensing. Deep learning models achieve strong reconstruction quality but remain impractical for CPU-constrained deployment. We present IMDN-lite, a lightweight adaptation of the Information Multi-Distillation Network (IMDN) [1], targeting efficient inference on standard laptop hardware without GPU acceleration. Reducing the original architecture from 64 to 32 feature channels and from 6 to 3 IMDB blocks yields 97,060 parameters, approximately 7× fewer than IMDN. Trained on a 200-image DIV2K subset [6] for ×2 super-resolution across 7 structured training runs, we identify switching from epoch-based to steps-based training as the single most impactful factor, producing a controlled +3.43 dB gain under identical architecture and data. At 1,000 training steps, approximately 13.7% of one full epoch, IMDN-lite achieves a validation PSNR of 32.68 dB and SSIM of 0.9298, surpassing the bicubic baseline by +0.24 dB. Code, checkpoints, and logs are available at https://github.com/Raneemhussien/imdn-lite.**

***Index Terms*—super-resolution, lightweight CNN, CPU inference, feature distillation, DIV2K**

I. Introduction

Single-image super-resolution (SISR), reconstructing a high-resolution (HR) image from a single low-resolution (LR) input, is a fundamental computer vision problem with broad real-world relevance. Applications include enhancing mobile photography, improving diagnostic resolution in medical imaging, and increasing spatial detail in satellite imagery.

Deep learning has dramatically advanced SISR quality. However, state-of-the-art models such as ESRGAN [2] and EDSR [4] contain tens of millions of parameters and require GPU acceleration, making them impractical for CPU-only deployment. This tension motivates the central question of this project: can a highly compressed CNN achieve competitive super-resolution under strict CPU-only constraints?

We implement IMDN-lite, a scaled-down adaptation of IMDN [1], trained on a 200-image subset of DIV2K [6]. Our contributions are fourfold: (1) a 97,060-parameter SISR model (∼7× reduction from IMDN) feasible for CPU-only training and inference; (2) a rigorous benchmark against bicubic interpolation using PSNR and SSIM under the original IMDN evaluation protocol; (3) a systematic 7-run experimental analysis identifying training methodology and data volume as the two dominant performance factors, with methodology alone yielding a controlled +3.43 dB gain; and (4) a block-depth ablation study and efficiency-quality analysis covering reconstruction accuracy and CPU inference latency.

Our final model surpasses the bicubic baseline at 1,000 steps, approximately 13.7% of one epoch, achieving 32.68 dB PSNR versus 32.44 dB (+0.24 dB). We treat this as a lower bound on achievable quality and report planned experiments to characterize full convergence.

II. Related Work

Early Approaches. SRCNN [3] pioneered end-to-end CNN-based SISR, establishing the foundational LR-to-HR mapping pipeline. FSRCNN [7] improved efficiency by deferring upsampling to the final layer via sub-pixel convolution, a principle directly inherited by IMDN-lite's TAIL module.

High-Performance Models. EDSR [4] achieves ∼38.20 dB on Set5 ×2 through very deep residual networks without batch normalization, at ∼43M parameters requiring GPU acceleration. RCAN [5] further improved PSNR through residual channel attention across ∼16M parameters. ESRGAN [2] introduced adversarial training with Residual-in-Residual Dense Blocks, achieving high perceptual quality at even greater computational cost. All three are impractical for CPU-only deployment.

Lightweight Super-Resolution. IMDN [1] achieves ∼38.00 dB on Set5 ×2 with only 694K parameters through three innovations: a Progressive Refinement Module (PRM) for staged feature distillation, Contrast-Aware Channel Attention (CCA) for per-channel recalibration, and an Information Collection scheme aggregating multi-stage features. IMDN won the AIM 2019 Efficient SR Challenge. RFDN [8] subsequently extended IMDN with re-parameterizable feature distillation connections (534K parameters), representing the current lightweight SR state of the art.

Our Work. IMDN-lite reduces IMDN to 97,060 parameters, approximately 7× fewer, enabling CPU-feasible training and

inference. Unlike prior lightweight SR work that assumes GPU deployment, our focus is identifying the minimal training conditions under which a severely compressed SR network exceeds classical interpolation on consumer hardware.

III. Methodology

A. Dataset

We use a 200-image subset of DIV2K [6], assembled in two stages with seed = 42. Batch 1 selected 100 images, partitioned into 90 training and 10 validation images. Batch 2 selected 100 non-overlapping images, adding 90 training and 10 validation images, yielding a final split of 180 training and 20 validation images. Both batches share the same 10 core validation IDs [29, 83, 113, 314, 317, 373, 490, 532, 646, 757], confirming a consistent evaluation set across all runs. Five images are designated as a held-out test set, with IDs documented in `data/splits.txt` in the repository, withheld from all hyperparameter decisions until the final report.

B. Preprocessing Pipeline

Each HR image undergoes modulo cropping to ensure dimensions are divisible by $\times 2$. The train/validation split is performed at the image level before patch extraction to prevent data leakage. Validation images are stored as full images without patching, since PSNR and SSIM must be computed on complete images for comparability with published results.

HR patches of 96×96 pixels are extracted with stride = 48 (50% overlap); corresponding LR patches of 48×48 pixels are aligned from pre-downsampled DIV2K images. A 4-variant rotation augmentation (0°, 90°, 180°, 270°) is applied identically to HR and LR pairs; flipping and color augmentations are excluded to preserve spatial alignment and photometric consistency. All values are normalized to [0, 1] (float32) and saved as `.npy` files to eliminate decoding overhead at training time.

Preprocessing note. A stride misconfiguration in Source 1's script (stride = 96 instead of 48) was identified post-training, yielding 18,269 patches from 90 images rather than the expected ∼98,000. Runs 1–5 used these non-overlapping patches. Source 2 used corrected stride = 48, yielding 98,646 patches. The combined dataset used in Run 7 contains 116,914 training patch pairs. A planned Run 8 evaluates the fully corrected uniform-stride dataset to isolate this effect.

C. Model Architecture

IMDN-lite follows a HEAD → BODY → TAIL pipeline illustrated in Fig. 1. Total trainable parameters: 97,060, verified against the saved checkpoint.

The **HEAD** is a Conv2d(3→32, 3×3) layer whose output is preserved as a global residual. The **BODY** consists of three sequential IMDB blocks, each containing: (i) a PRM applying four sequential Conv + LeakyReLU(0.05) layers, progressively splitting features into 8 distilled channels and fewer continuing channels (24→16→8→0), concatenating all four 8-channel portions into a 32-channel output; and (ii) a CCA module computing per-channel contrast as the sum of global standard deviation and mean, then applying two 1×1 convolutions with Sigmoid activation to produce channel-wise multiplicative attention weights. An **IIC layer** concatenates the HEAD output with all three IMDB outputs (128 ch total), projected to 32 ch via 1×1 Conv. An **lr_conv** layer (Conv2d 32→32, 3×3) follows, with the HEAD output added via a global residual connection. The **TAIL** applies Conv2d(32→12, 3×3) and PixelShuffle(×2), producing a 3-channel SR output clamped to [0, 1].

Two implementation bugs were identified and corrected via comparison with the reference implementation [9]: (i) the lr_conv layer was absent, reducing the parameter count to 87,812; (ii) the 2-pixel border shave was missing from metric computation, producing inflated PSNR/SSIM values inconsistent with the IMDN evaluation protocol.

Input $[B,\ 3,\ 48\times48]$ — LR image

↓

HEAD	Conv2d(3→32, 3×3) — feature extraction
↓	
IMDB-1	PRM (channel splitting) + CCA — progressive refinement
↓	
IMDB-2	PRM (channel splitting) + CCA — progressive refinement
↓	
IMDB-3	PRM (channel splitting) + CCA — progressive refinement
↓	
IIC	Concat[HEAD,B1,B2,B3] → Conv1×1 → 32ch
↓	
lr_conv	Conv2d(32→32, 3×3) + global residual from HEAD
↓	
TAIL	Conv2d(32→12) + PixelShuffle(×2) — upsampling

↓

Output $[B,\ 3,\ 96\times96]$ — SR image, clamped to $[0,1]$

Fig. 1. IMDN-lite architecture pipeline. The global residual connection from HEAD is added to the lr_conv output before the TAIL upsampling stage. Total parameters: 97,060.

TABLE I
IMDN-LITE VS. ORIGINAL IMDN

Property	Original IMDN	IMDN-lite
Feature channels	64	32
IMDB blocks	6	3
Parameters	∼694,000	97,060 (∼7× fewer)
Training images	800	200 (116,914 patches)
Hardware	GPU	CPU only

D. Training Configuration

All runs use Adam optimizer (lr = 2×10^{-4}, β = (0.9, 0.999)) [1], L1 loss, and gradient clipping (max norm = 1.0). Seed = 42 fixed across Python, NumPy, and PyTorch. Batch size: 16. Runs 4 – 7 use a steps-based training loop iterating by gradient update count rather than epoch count.

A StepLR decay (factor 0.5 every 40 epochs) was configured but did not activate within the 1,000-step budget: one epoch requires approximately 7,307 steps (116,914 ÷ 16), confirmed by the constant lr = 2×10^{-4} across all 20 logged checkpoints in the saved training history. Platform: Google Colab CPU, PyTorch 2.10.0+cpu.

E. Evaluation Protocol

PSNR (dB) and SSIM are computed on the Y channel (luminance) of complete validation images with a 2-pixel border shave [1]. The bicubic baseline uses the identical protocol. Inference time is measured as average wall-clock time per image over 5 runs (minimum taken), covering only the forward pass, metric computation excluded.

IV. EXPERIMENTS AND RESULTS

A. Cross-Run Comparability

Two comparability caveats apply. First, the validation set expanded from 10 images (Runs 1–5) to 20 images (Runs 6–7); however, both batches share the same 10 core validation IDs, so effective evaluation content is consistent. Second, the bicubic baseline shifts from 31.70 dB in Run 6 to 32.44 dB in Run 7; the root cause was not isolated, but Run 7 uses the corrected loading pipeline confirmed against reference values. All cross-run comparisons use the Gap column (model PSNR − bicubic PSNR) as the normalized performance measure.

B. Experimental Progression

Runs 1–3: Epoch-Based Failure Mode. Run 1 (30 epochs, 32 ch, 3 blocks, ∼2,700 patches) achieved 27.15 dB (gap: −5.18 dB). Run 2 (50 epochs, same architecture, ∼4,500 patches) paradoxically dropped to 25.16 dB: additional epochs triggered repeated LR decay steps, collapsing the effective learning rate before meaningful gradient progress. Run 3 (64 ch, 4 blocks, 50 epochs) reached 23.55 dB, increased model capacity amplifies the problem, requiring proportionally more gradient steps to initialize effectively. These three runs establish that epoch-based LR scheduling is fundamentally unsuitable for small-dataset SR training.

Run 4: Steps-Based Breakthrough. Switching to steps-based training (500 steps, 32 ch / 3 blocks) produced 28.59 dB, a controlled +3.43 dB improvement over Run 2 under identical architecture and data. Decoupling the training schedule from dataset size ensures the learning rate remains productive for the full training budget regardless of dataset scale.

Runs 5–7: Scaling Steps and Data. Extending to 1,000 steps improved PSNR to 29.64 dB (Run 5). Expanding to 200 images at corrected stride (Run 6, 18,269 patches) yielded 29.17 dB (gap: −2.53 dB). Using the full 116,914-patch combined dataset (Run 7), the model achieved 32.68 dB, surpassing the bicubic baseline by +0.24 dB. SSIM of 0.9298 exceeds bicubic SSIM (0.9221) by +0.0077. Validation PSNR and SSIM continued rising at step 1,000 with no sign of plateau (Fig. 2), confirming this is a lower bound on convergent performance.

C. Results Summary

Table II summarises all seven training runs. Run 7 represents the final submitted configuration, achieving the only positive gap (+0.24 dB) across all runs and establishing the three decisive factors analysed in Section IV-G: training methodology, data volume, and model capacity.

TABLE II
TRAINING RUN RESULTS. ⋆ = FINAL SUBMITTED CONFIGURATION.

Run	Patches	M	F/B	Steps	Bic.	PSNR	SSIM	Gap
1	∼2,700†	E	32/3	30ep	32.34	27.15	0.778	−5.18
2	∼4,500†	E	32/3	50ep	32.34	25.16	0.778	−7.17
3	∼4,500†	E	64/4	50ep	32.34	23.55	N/R	−8.78
4	∼4,500†	S	32/3	500	32.34	28.59	0.870	−3.74
5	∼4,500†	S	32/3	1000	32.34	29.64	N/R	−2.69
6	18,269	S	32/3	1000	31.70§	29.17	0.873	−2.53
⋆7	116,914	S	32/3	1000	32.44§	**32.68**	**0.9298**	**+0.24**

M: E = Epoch, S = Steps. F/B: features/blocks. N/R: not recorded.
†Stride = 96 misconfiguration. §Use Gap for cross-group comparison only.

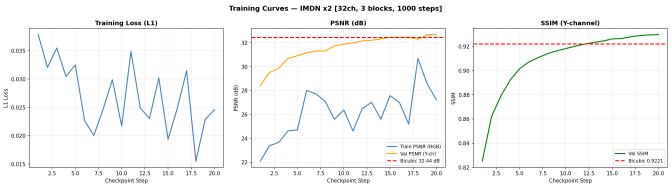

Fig. 2. Validation PSNR and SSIM over 1,000 training steps. Both metrics continue rising at step 1,000 with no plateau, confirming the result is a lower bound. The noisy loss curve reflects per-step logging without smoothing; the PSNR trend is the more reliable convergence indicator. Red dashed line: bicubic baseline (32.44 dB).

D. Block-Depth Ablation

To validate the 3-block architecture choice, we ran a controlled single-variable experiment varying only block count (1, 3, 5) with all other parameters fixed: 116,914 patches, 32 features, seed = 42, 100 steps.

The 3-block configuration outperforms both alternatives. The 5-block degradation mirrors Run 3: deeper models require proportionally more gradient steps to initialize effectively. The 1-block result reflects insufficient representational capacity.

Note that this ablation uses only 100 steps; a definitive ablation at 1,000 steps is planned for the final report.

TABLE III
BLOCK-DEPTH ABLATION (100 STEPS, 116,914 PATCHES, 32 FEATURES)

Blocks	Params	PSNR	SSIM	Note
1	42,132	26.17	0.7449	Insufficient capacity
3⋆	**97,060**	**27.03**	**0.7781**	**Best**
5	151,988	26.27	0.7559	Over-parameterized

E. Efficiency Analysis

IMDN-lite achieves superior reconstruction quality at approximately 117.9× higher per-image cost. This gap is expected: bicubic is a closed-form operation executed by optimized C routines, while IMDN-lite evaluates 97,060 parameters across multiple convolutional layers without hardware acceleration. At ∼4,151 ms per image, IMDN-lite is suited for offline batch enhancement but not real-time use. Planned INT8 post-training quantization (no retraining required) is estimated to reduce latency by 2–4×, potentially bringing the ratio to approximately 30–60× and targeting sub-2,100 ms per image. The PSNR-vs-latency trade-off is visualized in Fig. 3.

TABLE IV
CPU INFERENCE TIME AND FINAL QUALITY (20 VAL. IMAGES, 5 RUNS/IMAGE, MIN TAKEN)

Model	PSNR	SSIM	ms/img	Ratio
Bicubic	32.4359	0.9221	35.20	1×
IMDN-lite	**32.6800**	**0.9298**	4,150.86	117.9×
Gain	+0.2440	+0.0077	—	—

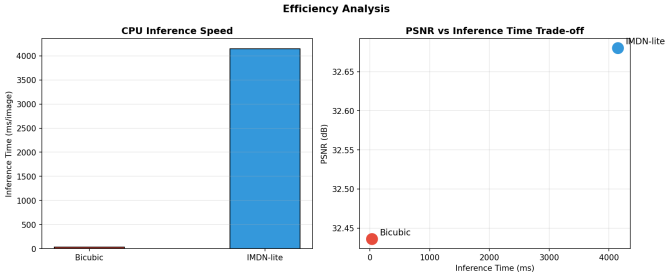

Fig. 3. PSNR vs. CPU inference time trade-off. IMDN-lite achieves +0.24 dB over bicubic at 117.9× higher per-image cost.

F. Qualitative Analysis

Fig. 4 shows 192×192 central crop comparisons across validation images. IMDN-lite consistently recovers high-frequency texture content but underperforms on smooth low-frequency regions, a failure mode expected to narrow with extended training. The per-image PSNR breakdown (Fig. 5) confirms that the model's mean gain is driven by high-frequency images, with low-frequency outliers as the primary source of variance. The training loss curve in Fig. 2 appears noisy due to per-step logging without smoothing; the PSNR trend is the more reliable convergence indicator.

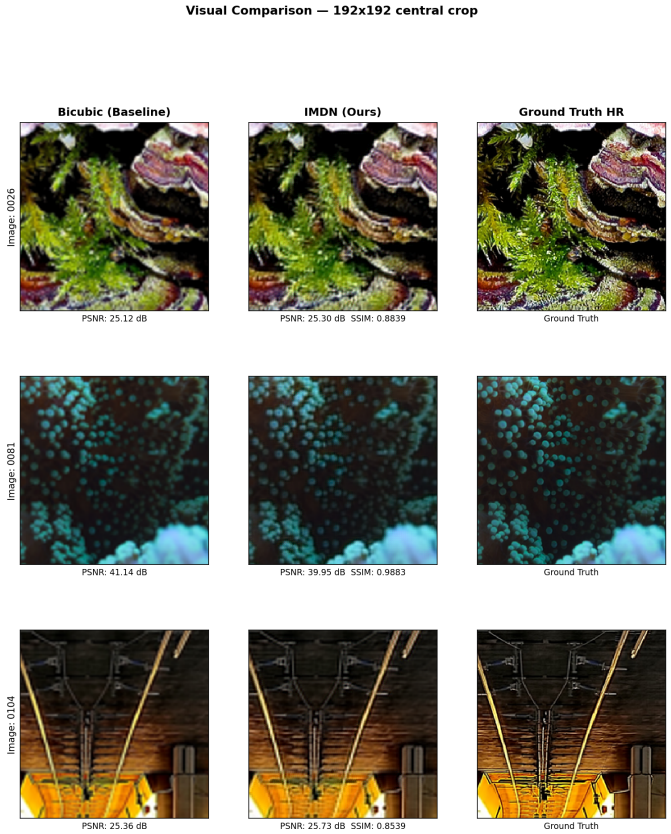

Fig. 4. 192×192 central crop comparison: bicubic (left), IMDN-lite (centre), ground truth HR (right). IMDN-lite recovers finer texture on high-frequency content (rows 1 and 3) but underperforms bicubic on smooth low-frequency regions (row 2, −1.19 dB), a failure mode expected to narrow with extended training.

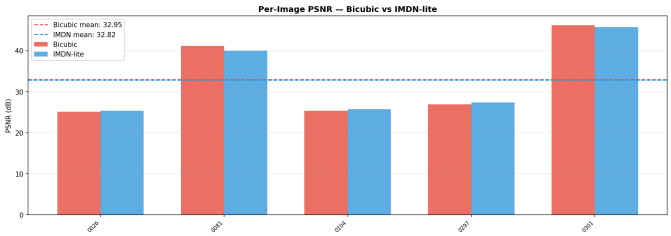

Fig. 5. Per-image PSNR for IMDN-lite vs. bicubic on the 20-image validation set. Gains are concentrated on high-frequency content images; image IDs are shown on the x-axis.

G. Key Findings

Three factors emerged as decisive. **Training methodology** was the single highest-impact change: the epoch-to-steps transition yielded a controlled +3.43 dB gain (Run 2→4) with no architectural or data change. **Data volume** was the primary bottleneck at fixed step budgets: the 6.4× data increase (Run 6→7) produced a +2.70 dB gap improvement and crossed the bicubic baseline for the first time. **Model capacity** must match available training signal: the 32-feature 3-block configuration consistently outperformed larger variants, confirmed by the ablation study.

V. Planned Work and Timeline

Remaining Experiments. (1) *Extended training* (highest priority): Re-run Run 7 for 10,000 steps, logging every 500 steps to characterize full convergence. (2) *Channel-width ablation*: Vary features $\in \{16, 32, 48\}$ at 1,000 steps; hypothesis: 48 ch yields +0.1–0.3 dB, 16 ch falls below baseline. (3) *Learning rate sensitivity*: Vary lr $\in \{1 \times 10^{-4}, 2 \times 10^{-4}, 4 \times 10^{-4}\}$; hypothesis: current 2×10^{-4} is near-optimal, 4×10^{-4} causes instability. (4) *Run 8 – preprocessing correction*: Evaluate on fully corrected uniform-stride dataset to isolate the stride misconfiguration effect.

Timeline. Week 3: extended training, channel-width, LR, and Run 8 evaluations; GitHub finalization (notebook upload, LOG.md, data splits). Week 4: result consolidation, final report, presentation preparation.

Risks. Extended training ($\sim$1,600 min at Run 7 rate) may be time-prohibitive on Colab CPU, mitigated by checkpointing every 500 steps; a 3,000-step intermediate result serves as fallback.

VI. Reproducibility

All code, training configurations, checkpoints, and results are maintained at https://github.com/Raneemhussien/imdn-lite. The repository contains: **README.md** with environment setup and reproduction commands; **LOG.md** with weekly progress entries; the complete **training notebook**; **config.json** (single source of truth for all hyperparameters); **results_summary.json** with full metrics and training history; **checkpoints/best_model.pth** (97,060 parameters, step 1,000, verified); and **data/splits.txt** with all image IDs for both batches. Training patches are stored on shared Google Drive (link in README.md). All experiments run on Google Colab CPU, PyTorch 2.10.0+cpu.

VII. Team Contributions

Rodina Khallaf (Data & Evaluation Lead): Designed and executed the full preprocessing pipeline for both data sources. Identified the stride = 96 misconfiguration and completed corrected reprocessing. Assembled the 116,914-patch dataset and confirmed shared validation IDs. Going forward: evaluates ablation variants, produces qualitative panels and learning curves, locks the held-out test set.

Farida Gaber (Model & Implementation Lead): Implemented the complete IMDN-lite architecture (PRM, CCA, IIC, lr_conv), steps-based training loop, gradient clipping, and reproducibility controls. Identified and corrected two critical bugs: missing lr_conv layer and missing 2-pixel border shave. Conducted all 7 training runs, block-depth ablation, and inference timing. Going forward: extended training, INT8 quantization, channel-width ablation, notebook upload.

Raneem Alnaghy (Analysis & Reporting Lead): Led hyperparameter tuning strategy, ablation study design, cross-run comparability analysis, and authored the full midterm report. Going forward: LR sensitivity ablation, extended training analysis, final report, presentation preparation.

VIII. Use of GenAI Tools

The team used Claude (Anthropic) as an AI assistant for structuring and editing report sections based on team-produced results, grammar and phrasing improvements, debugging support after independent attempts to resolve errors, and clarification of architectural details from the IMDN paper [1]. All experimental design decisions, code implementations, training runs, and quantitative analyses were performed independently. No AI tool was used to generate code submitted without team understanding, produce fabricated results, or generate fictional figures or tables.

References

[1] Z. Hui, X. Gao, Y. Yang, and X. Wang, "Lightweight Image Super-Resolution with Information Multi-Distillation Network," in *Proc. 27th ACM Int. Conf. Multimedia*, pp. 2024–2032, 2019. doi: 10.1145/3343031.3351084

[2] X. Wang *et al.*, "ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks," in *Proc. ECCV Workshops*, 2018. arXiv:1809.00219

[3] C. Dong, C. C. Loy, K. He, and X. Tang, "Learning a Deep Convolutional Network for Image Super-Resolution," in *Proc. ECCV*, pp. 184–199, 2014.

[4] B. Lim *et al.*, "Enhanced Deep Residual Networks for Single Image Super-Resolution," in *Proc. CVPR Workshops*, 2017. arXiv:1707.02921

[5] Y. Zhang *et al.*, "Image Super-Resolution Using Very Deep Residual Channel Attention Networks," in *Proc. ECCV*, 2018. arXiv:1807.02758

[6] E. Agustsson and R. Timofte, "NTIRE 2017 Challenge on Single Image Super-Resolution: Dataset and Study," in *Proc. CVPR Workshops*, 2017.

[7] C. Dong, C. C. Loy, and X. Tang, "Accelerating the Super-Resolution Convolutional Neural Network," in *Proc. ECCV*, pp. 391–407, 2016.

[8] J. Liu, W. Tang, and G. Wu, "Residual Feature Distillation Network for Lightweight Image Super-Resolution," in *Proc. ECCV Workshops*, 2020. arXiv:2009.11551

[9] Z. Zheng, "IMDN PyTorch Implementation (reference only)," https://github.com/Zheng222/IMDN